

Лабораторная 7. Поиск подстрок: КМП, Рабин-Карп и Бойер-Мур

Что считается правильным ответом

Есть текст длины n и шаблон длины m . Нужно вернуть все начальные позиции совпадений, включая пересечения. Для текста `aaaaa` и шаблона `aaa` правильный ответ `[0,1,2]`. Если после совпадения перескочить сразу на m позиций, два ответа потеряются. В комплекте пустой шаблон совпадает во всех $n+1$ границах текста, а шаблон длиннее текста не имеет вхождений. Это явно закреплённый контракт.[1]

Все функции возвращают позиции и число сравнений символов. Счётчик включает предобработку шаблона у КМП и хорошего суффикса Бойера-Мура; сравнение хешей и индексов символьным сравнением не считается. Время включает полную предобработку каждого вызова и создание списка ответов.

Наивный поиск — полезный эталон

Совмещаем шаблон с каждой возможной позицией текста, сравниваем символы слева направо и после несовпадения сдвигаемся на 1. После полного совпадения тоже сдвигаемся на 1, чтобы не потерять пересечения. Худший пример — длинный текст из `a` и шаблон `a...ab`: на каждом сдвиге почти весь шаблон совпадает.

Худшее время $O(nm)$, дополнительная рабочая память $O(1)$, не считая результата. Преимущество — простота, поэтому он служит независимым эталоном для остальных алгоритмов. В тестах дополнительно используется `str.find` в цикле, который продолжает поиск с позиции предыдущего совпадения плюс 1.

КМП: не проверять уже известное

Префикс — начало строки; суффикс — её конец. Собственный префикс короче всей строки. Значение $p[i]$ равно длине наибольшего собственного префикса `pattern[0:i+1]`, который одновременно является её суффиксом.

Для `ababaca` префикс-функция `[0,0,1,2,3,0,1]`. У `ababa` самая длинная граница `aba`, длина 3. Это информация о том, какая часть шаблона уже могла совпасть после сдвига.

При построении p берём $j=p[i-1]$. Если новый символ совпадает с `pattern[j]`, граница удлиняется на 1. Если нет, пробуем следующую более короткую границу $j=p[j-1]$, не начиная проверку с нуля. Значение j увеличивается только постепенно, а суммарное число откатов ограничено предыдущими увеличениями, поэтому построение линейно.

В поиске j обозначает длину уже совпавшего префикса шаблона. При несовпадении откатываем j по p , но индекс текста назад не возвращаем. При полном совпадении сохраняем позицию и устанавливаем $j=p[m-1]$. Именно это сохраняет возможность перекрывающихся совпадений. Для `aaa` после полного совпадения j становится 2: последние два `a` уже подходят как начало следующего вхождения.

Предобработка $O(m)$, поиск $O(n)$, рабочая память $O(m)$. С учётом списка из z ответов время $O(n+m+z)$, память $O(m+z)$. Поскольку $z \leq n+1$, обычно пишут $O(n+m)$. КМП особенно удобен, когда нужна предсказуемая граница работы без зависимости от качества хеша.[2]

Рабин-Карп: отпечаток скользящего окна

Полиномиальный хеш длины m вычисляется как последовательное $h = (h \cdot \text{base} + \text{code}(\text{symbol})) \bmod \text{modulus}$. При сдвиге окна вычитаем вклад уходящего символа, домножаем остаток на base и прибавляем новый символ. Предварительно вычисленная $\text{base}^{(m-1)} \bmod \text{modulus}$ позволяет обновить хеш за $O(1)$ арифметических операций.

Если хеши различаются, строки точно различны. Обратное неверно: разных строк бесконечно много, а остатков только modulus . Равенство хешей — лишь повод сравнить символы. Код обязательно делает такую проверку, поэтому коллизии ухудшают скорость, но не правильность.

Наглядная искусственная коллизия: при $\text{base}=1$ строки ab и ba дают одинаковую сумму кодов и одинаковый хеш. Тест ищет ab в ba и требует пустой ответ. Без посимвольной проверки получился бы ложный индекс 0.

При редких совпадениях хешей время близко к $O(n+m)$; более явно — $O(n+m+c \cdot m)$, где c — число окон с совпавшим хешем. Много истинных вхождений тоже требует проверки, даже если коллизий нет. Поэтому худший случай $O(nm)$, а фразу «Рабин-Карп всегда линейный» использовать нельзя. Дополнительная рабочая память $O(1)$, не считая ответа и хранения входных строк.

Бойер-Мур: сравнение справа налево

Шаблон сравнивается с текущим окном с правого конца. При несовпадении можно часто перескочить несколько позиций. Первая эвристика использует плохой символ текста: если символ не встречается в шаблоне, окно можно сдвинуть за его позицию. Если встречается, пытаемся совместить его с последним подходящим вхождением в шаблоне. Формула с глобальной последней позицией даёт $\max(1, j - \text{last}[\text{char}])$; отрицательный сдвиг запрещён.

Вторая эвристика использует уже совпавший хороший суффикс. Его пытаются совместить с другим вхождением внутри шаблона, которому предшествует другой символ, либо с подходящим префиксом. Так учитывается полезная информация, полученная до несовпадения. После полного совпадения таблица задаёт сдвиг, сохраняющий возможное перекрытие.

В реализации `good_suffix_table` строит таблицу `strong good suffix` за $O(m)$ с помощью границ суффиксов. `shifts[j+1]` применяется при несовпадении на позиции j , `shifts[0]` — после полного совпадения. Выбирается максимум безопасных сдвигов плохого символа и хорошего суффикса. Индекс сдвига увеличивается хотя бы на 1, поэтому заикливание исключено.

Предобработка занимает $O(m)$, рабочая память $O(m+\sigma)$, где σ — число различных символов шаблона. На большом алфавите и длинном шаблоне часто можно просматривать меньше символов текста, чем n . Это среднее наблюдаемое поведение, а не обещание для любого входа. Наша реализация поиска всех совпадений не содержит правила Галиля: на повторяющихся строках она может снова проверять длинные совпавшие участки, и безопасная худшая оценка остаётся $O(nm)$.

Алфавит и смысл эксперимента

В малом алфавите случайные совпадения коротких фрагментов чаще, а символ почти всегда присутствует в шаблоне, поэтому плохой символ даёт меньший сдвиг. КМП переиспользует совпавшие префиксы, Рабин-Карп может чаще проверять кандидатов, Бойер-Мур сильнее зависит от структуры шаблона.

«Естественно-подобный» текст генератора — синтетические слова из слогов, а не настоящий русский или английский корпус. Наблюдения нельзя автоматически переносить на любые реальные документы. Измерения выполнены на двух текстах примерно по миллиону символов и десяти шаблонах варианта 15. Все шаблоны выбраны из соответствующего текста и гарантированно встречаются.

```
python -m pytest labs/lab07 -q
python -m labs.lab07.experiment --variant 15
```

Сравниваются пять реализаций: наивная, КМП, Рабин-Карп, Бойер-Мур только с плохим символом и с обеими эвристиками. CSV хранит длину и сам шаблон, число совпадений, сравнения и пять временных повторов. На графиках длины шаблонов рассматриваются вместе с их содержимым: это разные шаблоны, а не чистый опыт, где меняется только m .

Проверка и вопросы на защите

Помимо сотен случайных строк проверены все бинарные тексты длиной до 6 и шаблоны длиной до 4 для обеих версий Бойера-Мура. Есть Unicode, пробелы, пустые строки, длинный шаблон, перекрытия и принудительная коллизия Рабина-Карпа.

- 1. Что означает $p[i]$?** Длину максимальной границы префикса, заканчивающегося на i : собственного префикса, совпадающего с суффиксом. Это длина, не индекс начала найденной подстроки.
- 2. Зачем сравнивать символы после равенства хешей?** Коллизии возможны всегда при конечном модуле. Без проверки алгоритм может вернуть несуществующее вхождение. Большой модуль уменьшает вероятность, но не заменяет проверку.
- 3. Как плохой символ уменьшает число сравнений?** Несовпавший символ указывает, какие положения шаблона заведомо невозможны. На большом алфавите он часто вообще отсутствует в шаблоне, позволяя длинный сдвиг.
- 4. Как влияет размер алфавита?** Малый алфавит повышает повторяемость и обычно сокращает безопасные сдвиги. Конкретный результат зависит также от длины, периодичности и числа истинных вхождений; это нужно сверять с таблицей опыта.
- 5. Как КМП сохраняет перекрытия?** После совпадения не обнуляет j , а сохраняет длину максимальной границы $p[m-1]$. Совпавший суффикс уже является началом следующего кандидата.
- 6. Зачем хорошая суффиксная эвристика?** Она использует не один несовпавший символ, а целый уже совпавший хвост. Иногда он позволяет больший безопасный сдвиг, чем плохой символ.
- 7. Какой алгоритм гарантирует линейный поиск здесь?** КМП. Рабин-Карп с проверкой кандидатов и реализованный Бойер-Мур без Галиля имеют $O(nm)$ в худшем случае при поиске всех совпадений.
- 8. Почему Рабин-Карп может делать мало сравнений, но работать медленно?** Хеш каждого окна требует арифметики по модулю. Она не входит в счётчик символов, но

занимает реальное время.

Источники

[1] МТУСИ. КИМ-07, 2026.

[2] Robert Sedgewick, Kevin Wayne. Substring Search, материалы авторов Algorithms, 4th edition.

Отчёт по ЛР 7. Поиск подстрок

Цель и выполненная работа

Реализованы алгоритмы и проверки, описанные в конспекте. Эксперимент выполнен на данных варианта 15, seed 45. В таблицах приведены фактические результаты запуска; время указано в секундах, если не оговорено иное.

Условия и методика

CPU: 12th Gen Intel(R) Core(TM) i7-12650H. ОС: Windows-11-10.0.26200-SP0. Python 3.12.12, NumPy 2.5.3, Matplotlib 3.11.1. Дата и время UTC: 2026-09-10T18:30:43.820661+00:00.

Один прогрев и пять повторов на точку, итог — медиана `time.perf_counter`. Ввод, генерация и подготовка свежего состояния исключены из таймера. Фоновые процессы и питание не контролировались; задержки рабочего компьютера могут влиять на отдельные точки. Контрольные суммы данных находятся в `environment.json`.

Результаты и их объяснение

ID	Алгоритм	m	Вхождений	Сравнения	c
0	naive	9	1	1094572	0.25125
0	KMP	9	1	1087637	0.157646
0	Rabin-Karp	9	1	9	0.351163
0	BM bad	9	1	161867	0.0726441
0	BM both	9	1	155683	0.0704053
1	naive	14	1	1166574	0.257482
1	KMP	14	1	1142818	0.165448
1	Rabin-Karp	14	1	14	0.339294
1	BM bad	14	1	275883	0.114309
1	BM both	14	1	239979	0.0970108
2	naive	20	1	1094614	0.245755
2	KMP	20	1	1087580	0.158787
2	Rabin-Karp	20	1	20	0.353672
2	BM bad	20	1	96198	0.0444065
2	BM both	20	1	91859	0.0413286
3	naive	28	1	1166439	0.260302
3	KMP	28	1	1142832	0.166804
3	Rabin-Karp	28	1	28	0.351805
3	BM bad	28	1	296595	0.123885
3	BM both	28	1	219601	0.0925106
4	naive	13	1	1094460	0.24968
4	KMP	13	1	1087648	0.153646
4	Rabin-Karp	13	1	13	0.349236
4	BM bad	13	1	131187	0.0592313
4	BM both	13	1	129059	0.0570883
5	naive	4	400	1166763	0.259102
5	KMP	4	400	1143154	0.168002
5	Rabin-Karp	4	400	1600	0.334058
5	BM bad	4	400	417964	0.176185
5	BM both	4	400	378538	0.160852
6	naive	26	1	1048203	0.248278
6	KMP	26	1	1039693	0.15679
6	Rabin-Karp	26	1	26	0.348659

ID	Алгоритм	m	Вхождений	Сравнения	c
6	BM bad	26	1	72526	0.0323247
6	BM both	26	1	71939	0.0327898
7	naive	20	1	1167324	0.264327
7	KMP	20	1	1143568	0.16713
7	Rabin-Karp	20	1	20	0.351711
7	BM bad	20	1	210450	0.088769
7	BM both	20	1	151395	0.0621229
8	naive	27	1	1047951	0.249157
8	KMP	27	1	1039336	0.149411
8	Rabin-Karp	27	1	27	0.342595
8	BM bad	27	1	105076	0.0433509
8	BM both	27	1	99060	0.0412795
9	naive	9	1	1166134	0.25092
9	KMP	9	1	1139640	0.165035
9	Rabin-Karp	9	1	9	0.340804
9	BM bad	9	1	222982	0.0928932
9	BM both	9	1	210920	0.0638355

Чётные ID относятся к естественно-подобному тексту, нечётные — к малому алфавиту. Шаблон ID=5 длины 4 встречается 400 раз; остальные выбранные шаблоны — по одному разу. Все пять реализаций вернули одинаковые позиции.

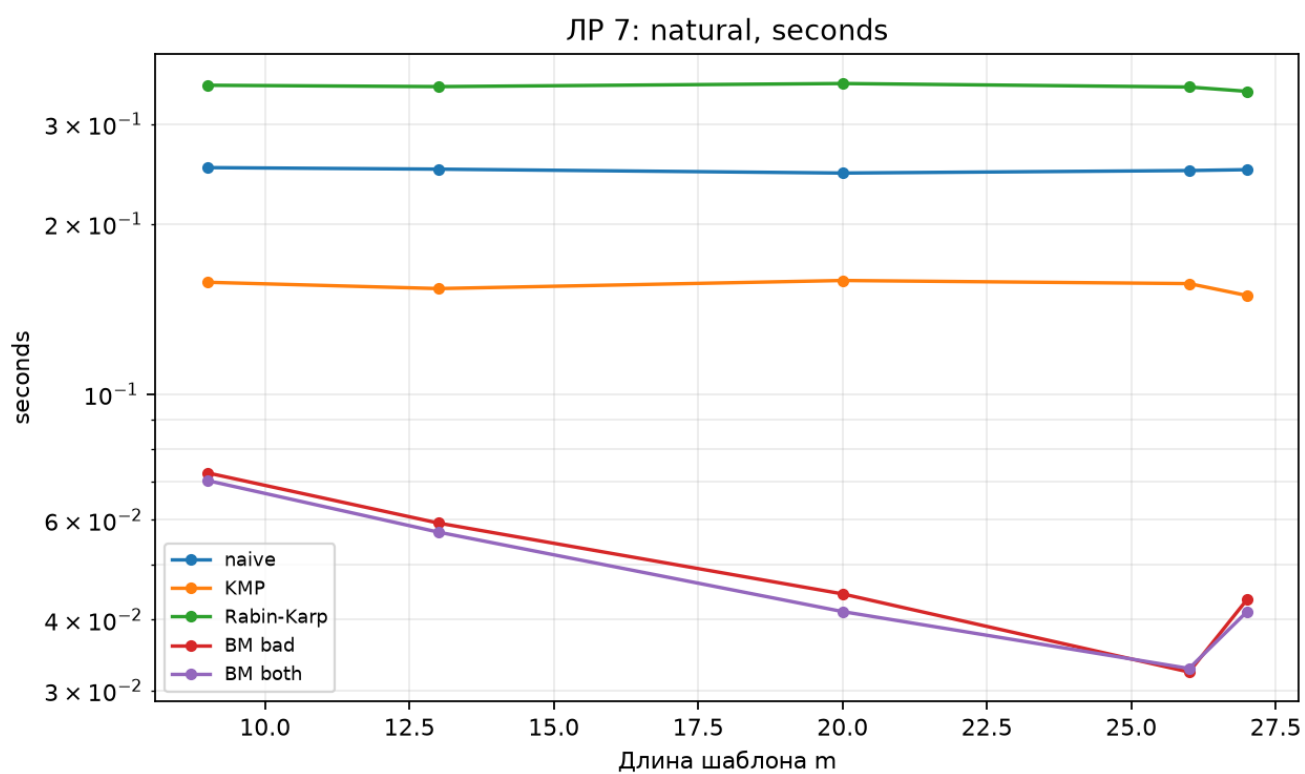
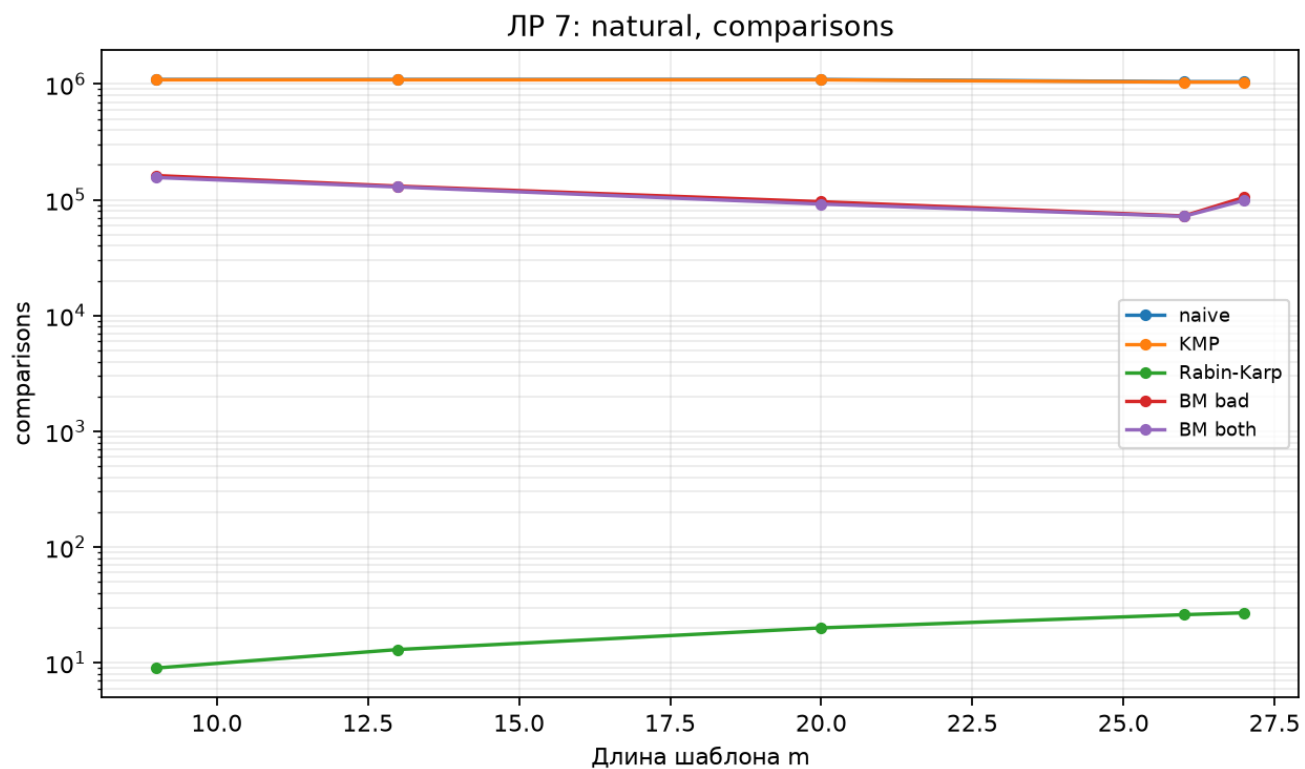
Бойер–Мур способен перескакивать через участки текста; КМП ограничивает повторные сравнения префиксной функцией. Рабин–Карп выполняет арифметику каждого окна даже при малом числе символьных сравнений. Хороший суффикс добавляет стоимость предобработки, поэтому его преимущество оценивается по реальным точкам. Эти десять шаблонов различаются и длиной, и содержимым: график не является изолированным измерением влияния одной только длины.

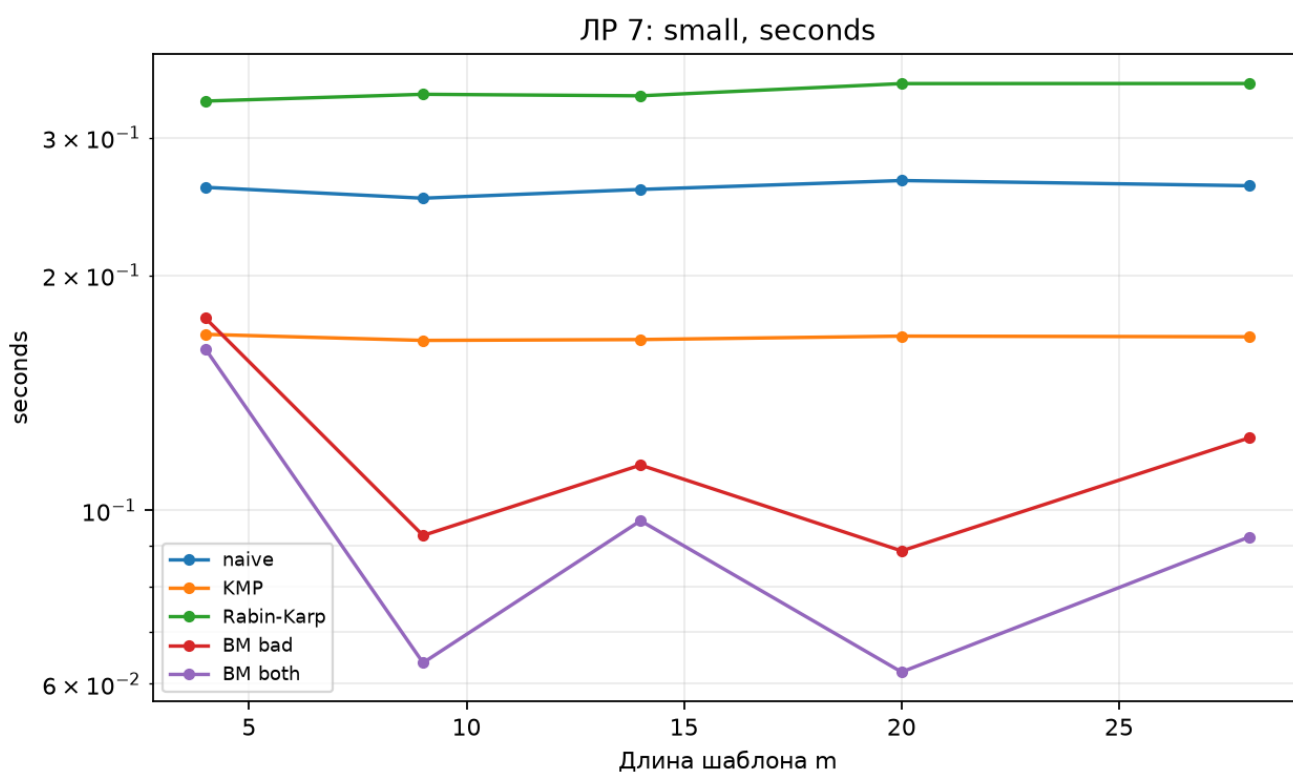
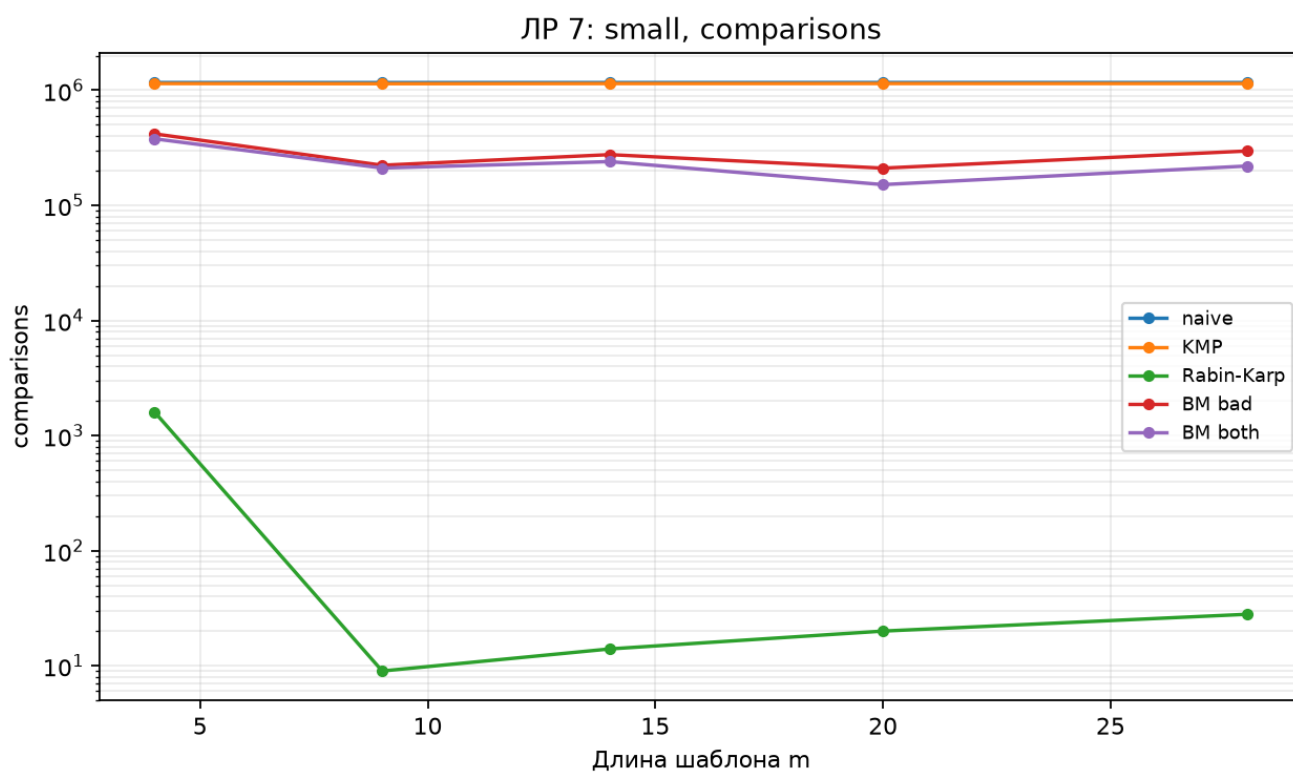
Проверка и вывод о применимости

Проверки находятся в test_lab.py. Запуск: `python -m pytest labs/lab07 -q`. Применены граничные случаи, инварианты и независимые эталоны; состав тестов подробно разобран в конспекте. Проверки всего комплекта также сохранены в verification.txt в корне репозитория.

Вывод: принять для учебного использования в рамках явно описанных контрактов. Измерения подтверждают основные тенденции роста; ограничения реализаций и сравнения указаны выше и в конспекте. Автоматическая проверка не оценивает готовность к устной защите.

Графики





Исходные данные отчёта

CSV содержит медиану и все пять длительностей trial_1...trial_5. Вспомогательные таблицы без времени содержат точные счётчики или свойства структуры.

- measurements.csv
- Паспорт окружения и контрольные суммы